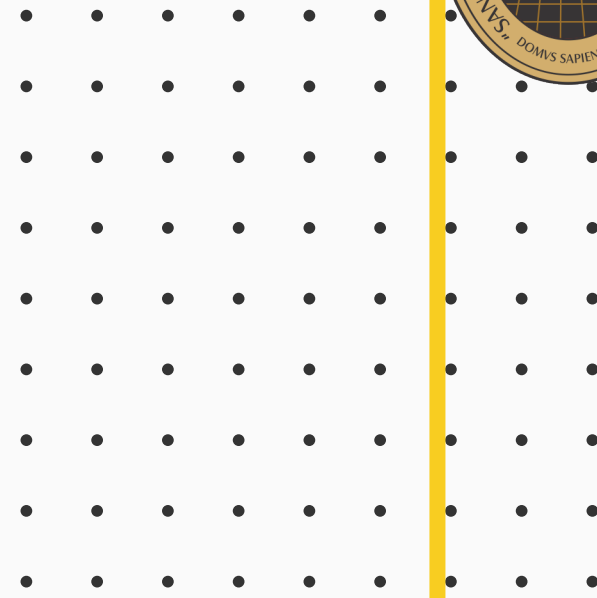




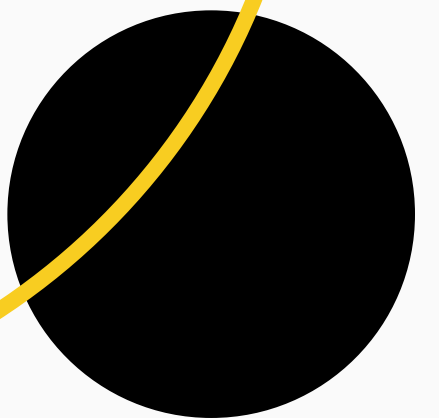
Universidad Nacional de San Agustín



SPONDYLUS BILLETERA

DIGITAL INTEROPERABLE • HITO 2

Fernandez Huarca, Rodrigo Alexander, Flores Choquehuanca,
Joe Daniel, Meza Vizcarra, Cielo Cristal, Nina Calizaya, Rafael
Diego, Quispe Condori, Alvaro Raul
UNSA | 06/07/2026



ENFOQUE Y CONTEXTO

Spondylus es un agregador de finanzas personales que consolida cuentas del consorcio de bancos e interactúa con la infraestructura **TAPP del BCRP**.

El proyecto adopta un enfoque riguroso:

- **Gestión de Requerimientos:** Guiado por **User Story Mapping** para entregas semanales de valor.
- **Lógica de Negocio:** Modelado formal mediante un grafo dirigido inmutable y contabilidad bitemporal de partida doble.
- **Desarrollo Ágil:** Reglas estrictas en Git, revisión por pares y diseño guiado por pruebas (TDD).

PROBLEMA Y PROPÓSITO

Eliminar la fragmentación de la información financiera y habilitar transferencias inmediatas e interoperables en un entorno distribuido y altamente consistente.

USER STORY MAPPING

El **Backbone** define las actividades esenciales del cliente de izquierda a derecha. Cada columna apila historias de usuario priorizadas en **releases** semanales.

1

Acceder al sistema
Autenticación federada: login con teléfono y PIN desde cualquier nodo bancario del consorcio.

2

Gestionar Cuentas
Visualización consolidada de saldos y cálculo del saldo real disponible (Safe-to-Spend).

3

Mover Dinero
Operaciones transaccionales distribuidas y consistentes (retiros, depósitos y transferencias).

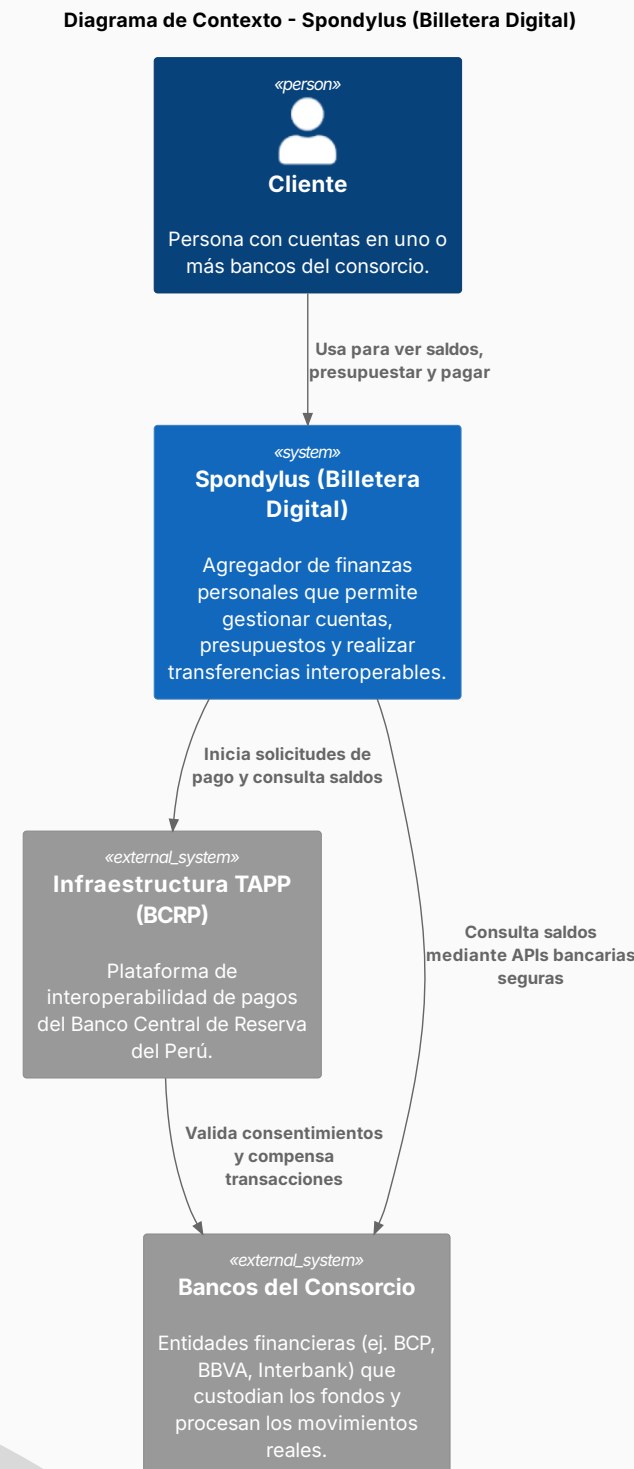
4

Control y Análisis
Presupuestos mensuales automáticos, corrección retroactiva de gastos y reporte consolidado por netting.

MODELO C4: DIAGRAMA DE CONTEXTO

El diagrama de contexto define los límites del sistema Spondylus y cómo se relaciona con actores y plataformas externas.

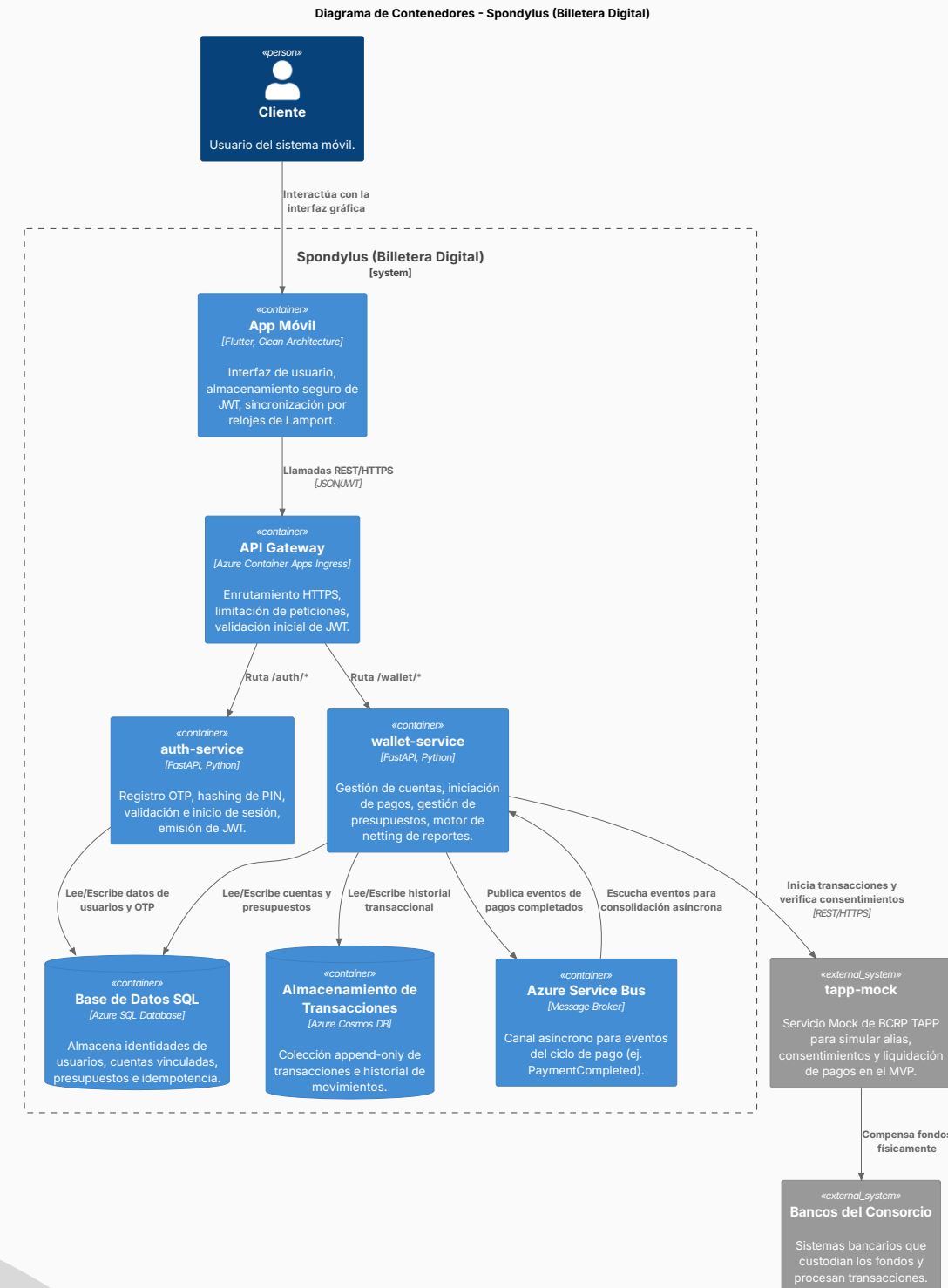
- **Cliente:** Actor principal. Inicia transferencias y visualiza reportes.
- **Infraestructura TAPP:** Pasarela de pagos del BCRP para liquidación interoperable.
- **Bancos del Consorcio:** Custodian los saldos reales de las cuentas del usuario.



MODELO C4: DIAGRAMA DE CONTENEDORES

Muestra las fronteras tecnológicas y la arquitectura de servicios distribuidos del MVP.

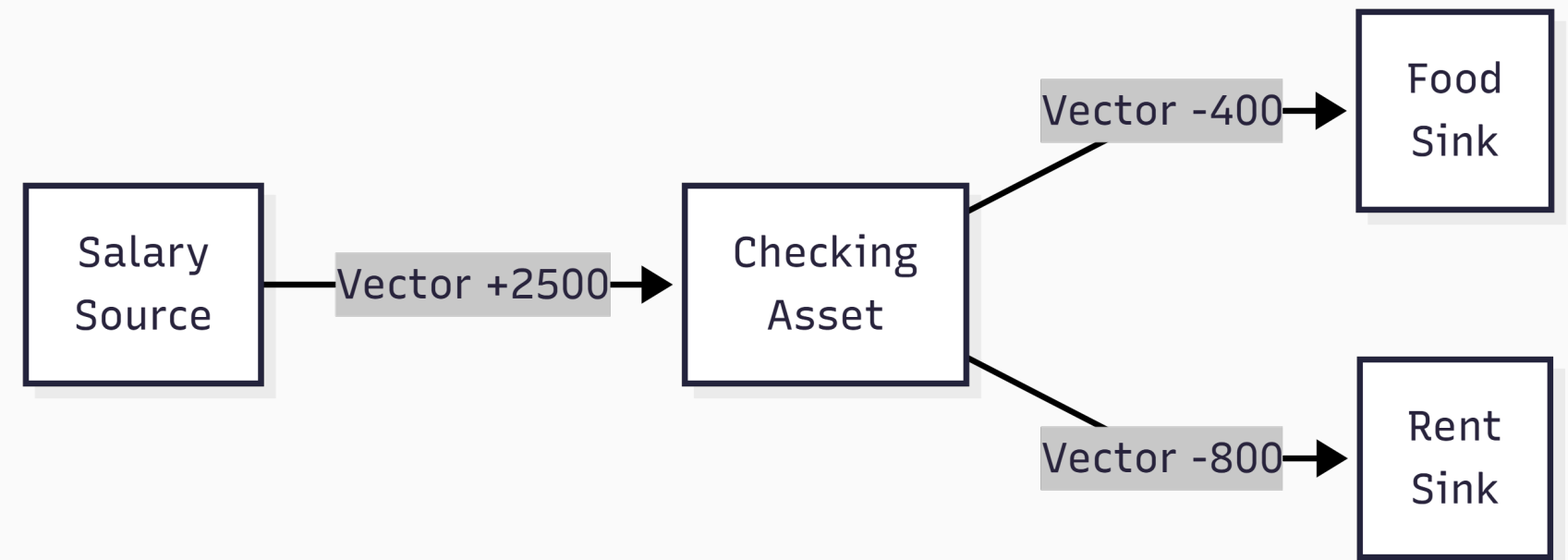
- **App Móvil:** Cliente en Flutter con sincronización de relojes de Lamport.
- **auth-service:** FastAPI. Maneja PIN y OTP de forma aislada.
- **wallet-service:** FastAPI. Contiene el core del ledger.
- **Bases de Datos:** Azure SQL para datos transaccionales y Cosmos DB para historial append-only.
- **Mensajería:** Azure Service Bus para el flujo asíncrono de confirmación.



BACKEND Y LÓGICA DE NEGOCIO

Implementado bajo una arquitectura hexagonal plana, desacoplando el núcleo de dominio de la infraestructura de almacenamiento.

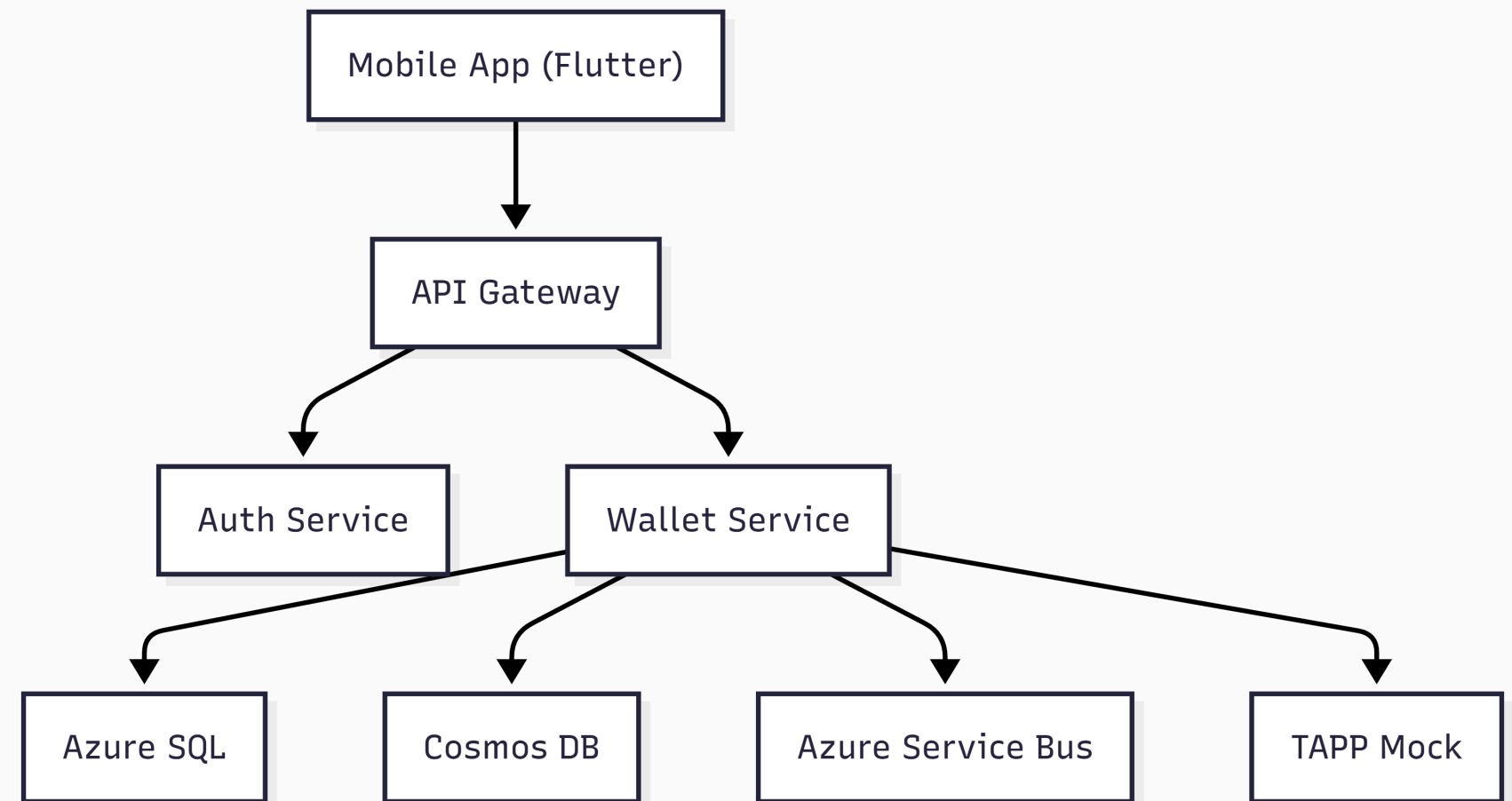
- **Modelo de Grafo Dirigido:** Las finanzas del cliente se representan mediante **Nodos** (Asset, Liability, Source, Sink) y **Vectores** (mutaciones de valor inmutables).
- **Ledger Append-Only:** Garantiza la trazabilidad e inmutabilidad requerida por la integración con TAPP del BCRP.
- **Motor de Netting:** Reduce y consolida movimientos lógicos utilizando un `lineage_token` único para reportes limpios libres de duplicación.
- **Safe-To-Spend:** Algoritmo que descuenta compromisos de presupuestos pasivos de los saldos líquidos reales para prevenir el sobregasto.



INFRAESTRUCTURA Y APP DISTRIBUIDA

Infraestructura elástica y automatizada diseñada para dar soporte al entorno distribuido de microservicios.

- **Azure Container Apps:** Servidor serverless para ejecutar contenedores FastAPI y mock-TAPP de manera segura y escalable sin la sobrecarga de AKS.
- **Infraestructura como Código (IaC):** Terraform automatiza el despliegue del API Gateway, bases de datos (SQL y Cosmos) y Key Vault.
- **Orquestación Local:** Archivo Docker Compose unificado para levantar de extremo a extremo los servicios y realizar pruebas de integración de manera ágil.
- **Simulación Externa:** Contenedor tapp-mock para aislar dependencias del BCRP y habilitar pruebas del ciclo de vida del pago.



APLICACIÓN MÓVIL (FLUTTER)

Cliente móvil desarrollado en Flutter aplicando Clean Architecture y estructurado por características (Feature-First).

- **Capas de la Arquitectura:** Separación estricta entre la capa de Presentación (widgets y Riverpod), Dominio (entidades y contratos) y Datos (orígenes y modelos Freezed).
- **Sincronización por Lamport:** Integración de `ClockInterceptor` en Dio para mantener la consistencia temporal lógica en peticiones distribuidas.
- **Pruebas Co-localizadas:** Pruebas ubicadas junto al código que testean (ej. `user_test.dart`) garantizando alta cohesión y mantenibilidad.
- **Módulos Implementados:** Login con PIN y SMS (auth), listado de cuentas (accounts), inicio de transferencias (transactions) y presupuestos (budget).

Two rounded rectangular boxes representing mobile app screens. The left box contains the text '[Pega aquí captura de:nAuth / Cuentas]' and the right box contains the text '[Pega aquí captura de:nTransacciones]'. Both boxes have a small grey pill-shaped notch at the top center.

[Pega aquí captura de:nAuth /
Cuentas]

[Pega aquí captura
de:nTransacciones]

ESTADO DE AVANCE Y ROADMAP



Fundación

Autenticación federada de usuarios,
cuentas locales y lectura distribuida.
(100% Completado)



Dinero Distribuido

Transferencias atómicas entre bancos
(protocolo 2PC), control de
conurrencia e idempotencia. (En
Progreso)



Control de Gastos

Presupuestos auto-renovables,
categorización retroactiva y netting
engine para reportes. (Planificado)



Cierre Académico

Pruebas de carga distribuidas, ajustes
de infraestructura Azure y
documentación final. (Planificado)

REFERENCIAS (IEEE)

[1] S. Brown, *Software Architecture for Developers: Visualise, Document and Explore Your Software Architecture*, 2018.

[2] M. Fowler, *Patterns of Enterprise Application Architecture*, Boston, MA: Addison-Wesley, 2002.

[3] J. Patton, *User Story Mapping: Discover the Whole Story, Build the Right Product*, Sebastopol, CA: O'Reilly Media, 2014.

[4] M. T. Özsu and P. Valduriez, *Principles of Distributed Database Systems*, 4th ed., Springer, 2020.

[5] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*, Boston, MA: Addison-Wesley, 2003.

[6] Banco Central de Reserva del Perú, *Reglamento de Interoperabilidad de los Servicios de Pago*, Lima, Perú, 2022.

